



The U-boot bootloader



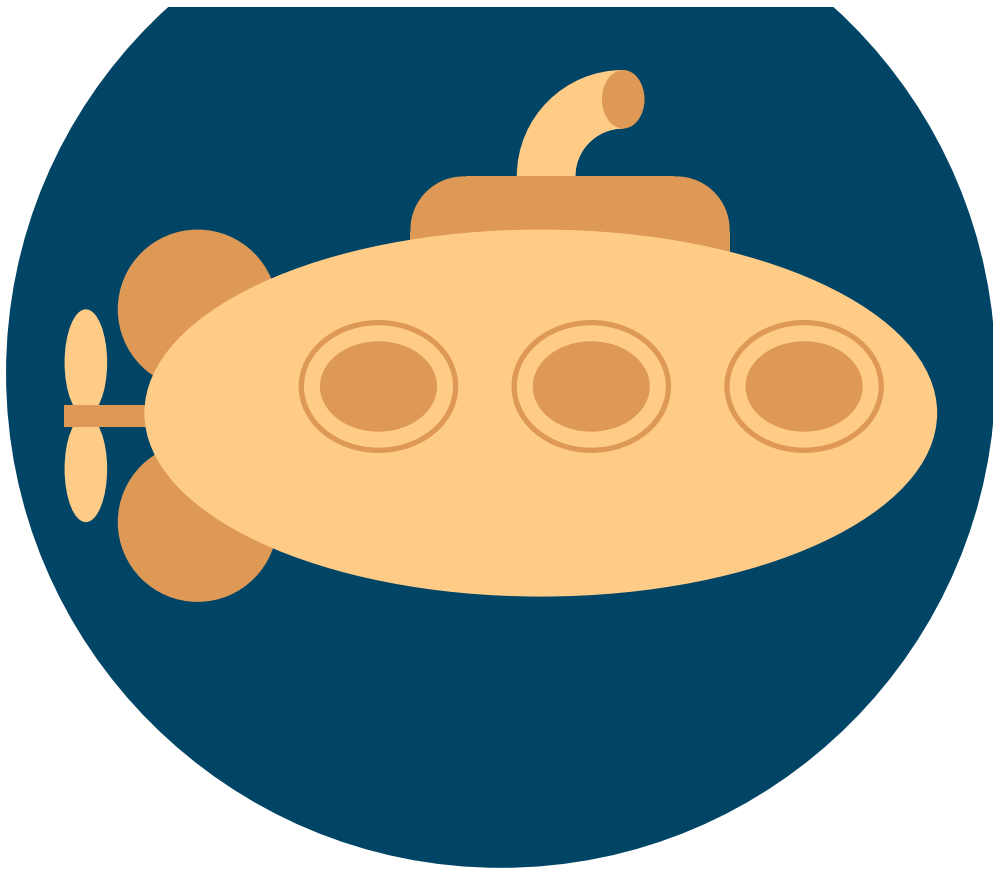
U-Boot

U-Boot is a typical free software project

- ▶ License: GPLv2 (same as Linux)
- ▶ Freely available at <https://www.denx.de/wiki/U-Boot>
- ▶ Documentation available at <https://u-boot.readthedocs.io/en/latest/>
- ▶ The latest development source code is available in a Git repository: <https://gitlab.denx.de/u-boot/u-boot>
- ▶ Development and discussions happen around an open mailing-list <https://lists.denx.de/pipermail/u-boot/>
- ▶ Follows a regular release schedule.
Every 2 or 3 months, a new version is released. Versions are named [YYYY.MM.](#)



U-Boot



U-Boot



U-Boot

Image source



Where to get U-Boot from?

- ▶ **Ideal:** your platform is supported directly by **upstream** U-Boot
 - Best quality \$\$_code reviewed and approved by the community
 - Long-term maintenance
 - Use directly U-Boot from <https://gitlab.denx.de/u-boot/u-boot> Git repository
- ▶ **Less ideal:** use a **fork** of U-Boot by your silicon vendor, system-on-module vendor or board vendor
 - Generally older, does not follow all upstream U-Boot updates
 - Changes not reviewed by the community \$\$_quality is often dubious
 - Check your HW vendor documentation/SDK
- ▶ If designing your own custom board
 - You will have to port U-Boot
 - If good support for your SoC in upstream U-Boot \$\$_use upstream U-Boot
 - If not \$\$_use the U-Boot fork from your SoC vendor



U-Boot configuration

- ▶ Configuration system based on *kconfig* from the Linux kernel
- ▶ The [configs/](#) directory contains configuration files for supported boards or platforms
 - There may be a single configuration supporting multiple boards based on the same processor
 - The configuration files defines all relevant options: CPU type, drivers needed, U-Boot features to compile in
 - Examples:
 - [configs/stm32mp15_basic_defconfig](#)
 - [configs/stm32mp15_trusted_defconfig](#)
- ▶ Note: migration to *kconfig* is still on-going
 - Not all boards have been converted to the new configuration system.
 - Many boards still have a combination of configuration settings in [include/configs/](#) header files, and configuration settings in [defconfig](#) files



U-Boot configuration file: [stm32mp15_trusted_defconfig](#)

```
CONFIG_ARM=y CONFIG_ARCH_STM32MP=y CONFIG_TFABOOT=y CONFIG_SYS_MALLOC_F_LEN=0x3000
CONFIG_ENV_OFFSET=0x280000
CONFIG_ENV_SECT_SIZE=0x40000
CONFIG_DEFAULT_DEVICE_TREE="stm32mp157c-ev1"
[...]
CONFIG_CMD_ADIMG=y CONFIG_CMD_ERASEENV=y CONFIG_CMD_NVEDIT_EFI=y CONFIG_CMD_MEMINFO=y CONFIG_CMD_MEMTEST=y
CONFIG_CMD_UNZIP=y CONFIG_CMD_ADC=y CONFIG_CMD_CLK=y CONFIG_CMD_DFU=y CONFIG_CMD_FUSE=y CONFIG_CMD_GPIO=y
[...]
CONFIG_SPI=y CONFIG_DM_SPI=y CONFIG_STM32_QSPI=y CONFIG_STM32_SPI=y
[...]
```

See the full file: [configs/stm32mp15_trusted_defconfig](#)



U-Boot configuration

- ▶ U-Boot must be configured before being compiled
- ▶ Configuration stored in a `.config` file
- ▶ Load a pre-defined configuration

```
\$_make BOARDNAME_defconfig
```

Where `BOARDNAME` is the name of a configuration, as visible in the `configs/` directory.

- ▶ You can then run `make menuconfig` to further customize U-Boot's configuration.



U-Boot configuration

```
U-Boot 2022.07 Configuration

Arrow keys navigate the menu. <Enter> selects submenus ---> (or empty submenu
----). Highlighted letters are hotkeys. Pressing <Y> includes, <N> excludes, <M>
modularizes features. Press <Esc><Esc> to exit, <?> for Help, </> for Search.
Legend: [*] built-in [ ] excluded <M> module < > module capable

*** Compiler: gcc (GCC) 12.1.1 20220507 (Red Hat 12.1.1-1) ***
Architecture select (ARM architecture) --->
[ ] Skip the calls to certain low level initialization functions
[ ] Skip the call to lowlevel_init during early boot ONLY
ARM architecture --->
General setup --->
API --->
Boot options --->
Console --->
Logging --->
Init options --->
Security support --->
Update support --->
Blob list --->
SPL / TPL / VPL --->
[*] FDT tools for simplefb support
Command line interface --->
Partition Types --->
Device Tree Control --->
Environment --->
[*] Networking support --->
(4) Number of receive packet buffers
Device Drivers --->
File systems --->
Library routines --->
[ ] Unit tests ----
[ ] Unit tests in SPL
Tools options --->

<Select> < Exit > < Help > < Save > < Load >
```



U-Boot compilation

- ▶ The path to the cross-compiler must be specified in the `CROSS_COMPILE` variable
- ▶ `CROSS_COMPILE` contains the prefix common to all cross-compilation tools, e.g `arm-linux-`
- ▶ Common to add the cross-compiler location in `PATH` to keep the `CROSS_COMPILE` value short

```
\$_export PATH=/path/to/toolchain/bin:\$PATH  
\$_make CROSS_COMPILE=arm-linux-
```

- ▶ The main result is a `u-boot.bin` file, which is the U-Boot image.
- ▶ Depending on your specific platform, or what storage device you're booting from (NAND or MMC), there may be other specialized images: `u-boot.img`, `u-boot.kwb...`



Concept of U-Boot SPL

- ▶ To meet the need of a two-stage boot process, U-Boot has the concept of *U-Boot SPL*
- ▶ SPL = *Secondary Program Loader*
- ▶ The SPL is a stripped-down version of U-Boot, made small enough to meet the size constraints of a first stage bootloader
- ▶ Configured through *menuconfig*, one can define the subset of drivers to include
- ▶ No U-Boot shell/commands: the behavior is hardcoded in C code
- ▶ For some platforms: TPL, *Tertiary Program Loader*, an even more minimal first stage bootloader to do TPL \$\$_SPL \$\$_main U-Boot.



Device Tree in U-Boot

- ▶ The *Device Tree* is a data structure that describes the topology of the hardware
- ▶ Allows software to know which hardware peripherals are available and how they are connected to the system
- ▶ Initially mainly used by Linux, now also used by U-Boot, Barebox, TF-A, etc.
- ▶ Used by U-Boot on most platforms.
- ▶ Since v2024.07 the Device Tree files location depends on `CONFIG_OF_UPSTREAM`:
 - `dtb/upstream/src/ARCH/VENDOR` when `CONFIG_OF_UPSTREAM` is set
 - `arch/ARCH/dts` otherwise
- ▶ One `.dts` for each board: need to create one if you build a custom board
- ▶ U-Boot *defconfigs* usually specify a default Device Tree, but it can be overridden using the `DEVICE_TREE` variable
- ▶ More details on the *Device Tree* later in this course.



U-Boot build example: TI AM335x BeagleBoneBlack

wireless

- ▶ One *defconfig* file suitable for all AM335x platforms: [configs/am335x_evm_defconfig](#)
 - Yes its name looks like it supports only the EVM (EValuation Module) board
 - Contains `CONFIG_DEFAULT_DEVICE_TREE="am335x-evm"` `$_uses` [arch/arm/dts/am335x-evm.dts](#) by default
- ▶ One *Device Tree* file describing the BeagleBoneBlack Wireless: [arch/arm/dts/am335x-boneblack-wireless.dts](#)
- ▶ Configure and build U-Boot

```
\$_export PATH=/path/to/toolchain/bin:\$PATH
\$_make am335x_evm_defconfig
\$_make DEVICE_TREE=am335x-boneblack-wireless CROSS_COMPILE=arm-linux-
```

- ▶ Produces:
 - [MLO](#), the SPL, first-stage bootloader. Called MLO (*Mmc LOad*) as required on TI platforms.
 - [u-boot.img](#), full U-Boot, second-stage bootloader



Installing U-Boot

1. If U-Boot is loaded from external storage, just update the binaries on such storage.
2. If U-Boot is loaded from internal storage (eMMC or NAND), you can update it using *Snagboot* (<https://github.com/bootlin/snagboot>) if it supports your SoC, or with the custom solution from the SoC vendor.
3. An alternative is to reflash internal storage with JTAG (if available), but that's more complicated and requires a JTAG probe.



U-boot shell prompt

- ▶ Connect the target to the host through a serial console.
- ▶ Power-up the board. On the serial console, you should see U-Boot starting up.
- ▶ The U-Boot shell offers a set of commands.
- ▶ The U-Boot shell is not a Linux shell: commands are completely different from Linux ones.

```
U-Boot SPL 2022.01 (Mar 31 2022 - 14:58:17 +0200)
Trying to boot from MMC1
```

```
U-Boot 2022.01 (Mar 31 2022 - 14:58:17 +0200)
```

```
CPU   : AM335X-GP rev 2.1
```

```
Model: TI AM335x BeagleBone Black DRAM:  512 MiB
```



U-boot shell prompt

```
WDT:   Started wdt@44e35000 with servicing (60s
timeout)
NAND:   0 MiB
MMC:   OMAP SD/MMC: 0, OMAP SD/MMC: 1
Loading Environment from FAT... OK
Net:   Could not get PHY for ethernet@4a100000: addr
0
eth2: ethernet@4a100000, eth3: usb_ether [PRIME]
Hit any key to stop autoboot:  0
=>
```



U-Boot `help` command

- ▶ `help` command to list all available commands
- ▶ The set of available commands depend on the U-Boot configuration
 - Many `CONFIG_CMD_*` options to enable commands at compile time
 - See *Command line interface* submenu in `menuconfig`
- ▶ `help <command>` for the complete help of one command

```
STM32MP> help
?          - alias for 'help'
adc        - ADC sub-system
adtimg     - manipulate
dtb/dtbo   Android image base    - print or set
address offset
[...]
usb        - USB sub-system
[...]
```



U-Boot **help** command

```
STM32MP> help usb usb - USB sub-system
```

Usage:

```
usb start - start (scan) USB controller  
usb reset - reset (rescan) USB controller  
usb stop [f] - stop USB [f]=force stop  
usb tree - show USB device tree  
usb info [dev] - show available USB devices  
[...]
```



U-Boot information commands

Version details: `version`

```
=> version U-Boot 2020.04 (May 26 2020 - 16:05:43
+0200)
arm-linux-gcc (crosstool-NG 1.24.0.105_5659366)
9.2.0
GNU ld (crosstool-NG 1.24.0.105_5659366) 2.34
```

NAND flash information: `nand info`

```
=> nand info Device 0: nand0, sector size 128 KiB
Page size      2048 b
OOB size       64 b
Erase size     131072 b
subpagesize    2048 b
options        0x40004200
bbt options    0x00008000
```

MMC information: `mmc info`

```
=> mmc info Device: STM32 SD/MMC
Manufacturer ID: 3
[...]
```



U-Boot information commands

Capacity: 14.8 GiB
Bus Width: 4-bit

Board information: `bdinfo`

```
=> bdinfo boot_params = 0x00000000
DRAM bank    = 0x00000000
-> start     = 0xc0000000
-> size      = 0x20000000
flashstart   = 0x00000000
flashsize    = 0x00000000
flashoffset  = 0x00000000
baudrate     = 115200 bps relocaddr   = 0xddb21000
reloc off    = 0x1da21000
[...]
fdt_blob     = 0xdbb01950
new_fdt      = 0xdbb01950
fdt_size     = 0x0001d540
Video        = display-controller@5a001000 inactive
[...]
```

- ▶ DRAM starts at `0xc0000000`, for a size of 512 MB (`0x20000000`).



U-Boot information commands

- ▶ The end of the memory is used by U-Boot itself: `relocaddr` is the location of U-Boot in RAM.



Concept of U-Boot environment

- ▶ A significant part of the U-Boot configuration happens at compile time: `menuconfig`
- ▶ U-Boot also has runtime configuration, through the concept of *environment variables*
- ▶ Environment variables are key/value pairs
 - Some specific environment variables impact the behavior of different U-Boot commands
 - Additional custom environment variables can be added, and used in *scripts*
- ▶ U-Boot environment variables are loaded and modified in RAM
- ▶ U-Boot has a default environment built into its binary
 - used when no other environment is found
 - defined in the configuration
 - the default environment is sometimes quite complex in some existing configurations
- ▶ The environment can be persistently stored in non-volatile storage



U-Boot environment persistent storage

Depending on the configuration, the U-Boot environment can be:

- ▶ At a fixed offset in NAND flash
- ▶ At a fixed offset on MMC or USB storage, before the beginning of the first partition.
- ▶ In a file on a FAT or ext4 partition
- ▶ In a UBI volume
- ▶ Not stored at all, only the built-in environment in the U-Boot binary is used



U-Boot environment persistent storage

.config - U-Boot 2020.07 Configuration

Environment

```
[ ] Environment is not stored
[ ] Environment in EEPROM
[*] Environment is in a FAT filesystem
[ ] Environment is in a EXT4 filesystem
[ ] Environment in flash memory
[ ] Environment in an MMC device
[ ] Environment in a NAND device
[ ] Environment in a non-volatile RAM
[ ] Environment is in OneNAND
[ ] Environment is in remote memory space
[ ] Environment in a UBI volume
(mmc) Name of the block device for the environment
(0) Device and partition for where to store the environment in FAT
(uboot.env) Name of the FAT file to use for the environment
(0x4000) Environment Size
[*] Relocate gd->env_addr
[ ] Create default environment from file
[ ] Add run-time information to the environment
[ ] Block forced environment operations
```

U-Boot environment configuration menu



U-Boot environment commands

- ▶ `printenv` Shows all variables
- ▶ `printenv <variable-name>` Shows the value of a variable
- ▶ `setenv <variable-name> <variable-value>` Changes the value of a variable or defines a new one, only in RAM
- ▶ `editenv <variable-name>` Interactively edits the value of a variable, only in RAM
- ▶ After an `editenv` or `setenv`, changes in the environment are lost if they are not saved persistently
- ▶ `saveenv` Saves the current state of the environment to storage for persistence.
- ▶ `env` command, with many sub-commands: `env default`, `env info`, `env erase`, `env set`, `env save`, etc.



U-Boot environment commands example

```
=> printenv baudrate=19200
ethaddr=00:40:95:36:35:33
netmask=255.255.255.0
ipaddr=10.0.0.11
serverip=10.0.0.1
stdin=serial stdout=serial stderr=serial
=> setenv serverip 10.0.0.100
=> printenv serverip serverip=10.0.0.100
=> saveenv
```



U-Boot memory allocation

- ▶ Many commands in U-Boot loading data into memory, or using data from memory, expect a RAM address as argument
- ▶ No built-in memory allocation mechanism \$\$_up to the user to know usable memory areas to load/use data
- ▶ Use the output of `bdinfo` to know the start address and size of RAM
- ▶ Avoid the end of the RAM, which is used by the U-Boot code and dynamic memory allocations
- ▶ Not the best part of the U-Boot design, sadly



U-Boot memory manipulation commands

- ▶ Commands to inspect or modify any memory location, useful for debugging, poking into hardware registers, etc.
- ▶ Addresses manipulated in U-Boot are directly physical addresses
- ▶ Memory display
- ▶ Memory write `mw [.b, .w, .l, .q] address value [count]`
- ▶ Memory modify (modify memory contents interactively starting from address) `mm [.b, .w, .l, .q] address`



U-Boot raw storage commands U-Boot can manipulate raw storage

devices:

▶ NAND flash

- `nand info`
- `nand read <addr> <off|partition> <size>`
- `nand erase [<off> [<size>]]`
- `nand write <addr> <off|partition> <size>`
- More: `help nand`

▶ MMC

- `mmc info`
-
-
- `mmc part` to show partition table
- `mmc dev` to show/set current MMC device



U-Boot raw storage commands U-Boot can manipulate raw storage

- More: `help mmc`
- ▶ USB storage
 - `usb info`
 -
 -
 - `usb part`
 - `usb dev`
 - More: `help usb`

Note: `<addr>` are addresses in RAM where data is stored



U-Boot commands example

List partitions on MMC

```
STM32MP> mmc part Partition Map for MMC device 0 --  
Partition Type: EFI
```

Part	Start LBA	End LBA	Name
	Attributes		
	Type GUID		
	Partition GUID		
1	0x00000022	0x000001d3	"fsbl1"
	attrs: 0x0000000000000000		
	type: 0fc63daf-8483-4772-8e79-3d69d8477de4		
	type: linux		
	guid: 72c63477-c475-4cf7-988e-b763bce4604e		
2	0x000001d4	0x00000385	"fsbl2"
	attrs: 0x0000000000000000		
	type: 0fc63daf-8483-4772-8e79-3d69d8477de4		
	type: linux		
	guid: 66d616db-de56-4a1e-9b13-9b1a5a6e360f		
3	0x00000386	0x00001385	"fip"
	attrs: 0x0000000000000000		
	type: 0fc63daf-8483-4772-8e79-3d69d8477de4		
	type: linux		



U-Boot commands example

```
guid: 6251ecf7-d985-4d81-a396-7a6b6fab8b7c  
[...]
```

Read block 0x22 from MMC to RAM

0xc0000000

```
STM32MP> mmc read c0000000 22 1
```

```
MMC read: dev #0, block #34, count 1 ... 1 blocks  
read: OK
```

Dump memory at 0xc0000000

```
STM32MP> md c0000000  
c0000000: 324d5453 00000000 00000000 00000000  
STM2.....  
c0000010: 00000000 00000000 00000000  
00000000 .....  
c0000020: 00000000 00000000 00000000  
00000000 .....  
c0000030: 00000000 00000000 00000000  
00000000 .....
```



U-Boot filesystem storage commands

- ▶ U-Boot has support for many filesystems
 - The exact list of supported filesystems depends on the U-Boot configuration
- ▶ Per-filesystem commands
 - FAT: `fatinfo`, `fatls`, `fatsize`, `fatload`, `fatwrite`
 - ext2/3/4: `ext2ls`, `ext4ls`, `ext2load`, `ext4load`, `ext4size`, `ext4write`
 - Squashfs: `sqfsls`, `sqfsload`
- ▶ “New” generic commands, working for all filesystem types
 - Load a file: `load <interface> [<dev[:part]> [<addr> [<filename> [bytes [pos]]]]]`
 - List files: `ls <interface> [<dev[:part]> [directory]]`
 - Get the size of a file: `size <interface> <dev[:part]> <filename>` (result stored in `filesize` environment variable)
 - `interface`: `mmc`, `usb`
 - `dev`: device number, `0` for first device, `1` for second device
 - `part`: partition number



U-Boot filesystem command example

List files

```
STM32MP> ls mmc 0:4
<DIR>      1024 .
<DIR>      1024 ..
<DIR>     12288 lost+found
<DIR>      2048 bin
<DIR>      1024 boot
<DIR>      1024 dev
<DIR>      1024 etc
[...]

STM32MP> ls mmc 0:4 /etc
<DIR>      1024 .
<DIR>      1024 ..
          209 asound.conf
<DIR>      1024 fonts
          334 fstab
          347 group
[...]
```

Load file



U-Boot filesystem command example

```
STM32MP> load mmc 0:4 c0000000 /etc/fstab
334 bytes read in 143 ms (2 KiB/s)
```

Show file contents

```
STM32MP> md c0000000
c0000000: 663c2023 20656c69 74737973 093e6d65
#<filesystem>.
c0000010: 756f6d3c 7020746e 3c093e74 65707974
<mountpt>.\<type c0000020: 6f3c093e 6f697470
093e736e 6d75643c >.<options>.\<dum c0000030:
3c093e70 73736170 642f0a3e 722f7665 p>.<pass>./dev/
r c0000040: 09746f6f 6509092f 09327478 6e2c7772
oot./..ext2.rw,n
[...]
```



U-Boot networking

▶ Environment variables

- `ethaddr`: MAC address
- `ipaddr`: IP address of the board
- `serverip`: IP address of the server for network related commands

▶ Important commands

- `ping`: ping a destination machine. Note: U-Boot is not an operating system with multitasking/interrupts, so ping from another machine to U-Boot cannot work.
- `tftp`: load a file using the TFTP protocol
- `dhcp`: get an IP address using DHCP



- ▶ Network transfer from the development workstation to U-Boot on the target takes place through TFTP
 - *Trivial File Transfer Protocol*
 - Somewhat similar to FTP, but without authentication and over UDP
- ▶ A TFTP server is needed on the development workstation
 - `sudo apt install tftpd-hpa`
 - All files in `/srv/tftp` are then visible through TFTP
 - A TFTP client is available in the `tftp-hpa` package, for testing
- ▶ A TFTP client is integrated into U-Boot
 - Configure the `ipaddr`, `serverip`, and `ethaddr` environment variables
 - Use `tftp <address> <filename>` to load file contents to the specified RAM address
 - Example: `tftp 0x21000000 zImage`



Scripts in environment variables

- ▶ Environment variables can contain small scripts, to execute several commands and test the results of commands.
 - Useful to automate booting or upgrade processes
 - Several commands can be chained using the `;` operator
 - Tests can be done using `if command ; then ... ; else ... ; fi`
 - Scripts are executed using `run <variable-name>`
 - You can reference other variables using `\$variable-name`
- ▶ Examples
 - `'setenv bootcmd tftp 0x21000000 zImage; tftp 0x22000000 dtb; bootz 0x21000000 - 0x22000000'`
 - `'setenv mmc-boot if fatload mmc 0 80000000 boot.ini; then source; else if fatload mmc 0 80000000 zImage; then run mmc-do-boot; fi; fi'`



U-Boot booting commands

- ▶ Commands to boot a Linux kernel image
 - `bootz` `$$_boot` a compressed ARM32 `zImage`
 - `booti` `$$_boot` an uncompressed ARM64 or RISC-V `Image`
 - `bootm` `$$_boot` a kernel image with legacy U-Boot headers
 - `zboot` `$$_boot` a compressed x86 `bzImage`
- ▶ `bootz [addr [initrd[:size]] [fdt]]`
 - `addr`: address of the kernel image in RAM
 - `initrd`: address of the *initrd* or *initramfs*, if any. Otherwise, must pass -
 - `fdt`: address of the *Device Tree* passed to the Linux kernel
- ▶ Important environment variables
 - `bootcmd`: list of commands executed automatically by U-Boot after the count down
 - `bootargs`: Linux kernel command line



U-Boot booting example

Load kernel image and Device Tree

```
STM32MP> ls mmc 0:4 /boot
<DIR>      1024 .
<DIR>      1024 ..
          117969 stm32mp157c-dk2.dtb
          7538376 zImage

STM32MP> load mmc 0:4 c2000000 /boot/zImage
7538376 bytes read in 463 ms (15.5 MiB/s)

STM32MP> load mmc 0:4 c4000000 /boot/stm32mp157c-dk2.dtb
117969 bytes read in 148 ms (778.3 KiB/s)
```

Set kernel command line and boot

```
STM32MP> setenv bootargs root=/dev/mmcblk0p4 rootwait

STM32MP> bootz c2000000 - c4000000
Kernel image @ 0xc2000000 [ 0x000000 - 0x7306c8 ]
#Flattened Device Tree blob at c4000000
  Booting using the fdt blob at 0xc4000000
  Loading Device Tree to cffe0000, end cffffcd0 ... OK
[...]
```



FIT image

- ▶ U-Boot has a concept of **FIT** image
- ▶ FIT = *Flat Image Tree*
- ▶ Container format that allows to bundle multiple images into one
 - Multiple kernel images
 - Multiple Device Trees
 - Multiple initramfs
 - Any other image: FPGA bitstream, etc.
- ▶ Typically useful for secure booting and to ensure binaries don't overlap in memory.
- ▶ Interestingly, relies on the *Device Tree Compiler*
 - `.its` file describes the contents of the image
 - Device Tree Compiler compiles it into an `.itb`
- ▶ U-Boot can load an `.itb` image and use its different elements
- ▶ <https://www.thegoodpenguin.co.uk/blog/u-boot-fit-image-overview/>



Generic Distro boot (1)

- ▶ Each board/platform used to have its own U-Boot environment, with custom variables/commands
- ▶ Wish to standardize the behavior of bootloaders, including U-Boot
- ▶ *Generic Distro boot* concept
- ▶ If enabled, at boot time, U-Boot:
 - Can be asked to locate a bootable partition (`part list` command), as defined by the bootable flag of the partition table
 - With the `sysboot` command, will look for a `/extlinux/extlinux.conf` or `/boot/extlinux/extlinux.conf` file describing how to boot, and will offer a prompt in the console to choose between available configurations.
 - Once a configuration is selected, will load and boot the corresponding kernel, device tree and initramfs images.
 - Example `bootcmd: part list mmc 0 -bootable bootpart; sysboot mmc 0:\$bootpart any`
- ▶ <https://u-boot.readthedocs.io/en/latest/develop/distro.html>



Generic Distro boot (2)

Several environment variables need to be set:

- ▶ `kernel_addr_r`: address in RAM to load the kernel image
- ▶ `ramdisk_addr_r`: address in RAM to load the initramfs image (if any)
- ▶ `fdt_addr_r`: address in RAM to load the DTB (Flattened Device Tree)
- ▶ `pxefile_addr_r`: address in RAM to load the configuration file (usually `extlinux.conf`)
- ▶ `bootfile`: the path to the configuration file, for example `/boot/extlinux/extlinux.conf`

Example `/boot/extlinux/extlinux.conf`

```
label stm32mp157c-dk2-buildroot
kernel /boot/zImage
devicetree /boot/stm32mp157c-dk2.dtb
append root=/dev/mmcblk0p4 rootwait
```

U-Boot boot log

```
Hit any key to stop autoboot: 0
Boot over mmc0!
switch to partitions #0, OK
mmc0 is current device Scanning mmc 0:4...
Found /boot/extlinux/extlinux.conf Retrieving
file: /boot/extlinux/extlinux.conf
131 bytes read in 143 ms (0 Bytes/s)
1:      stm32mp157c-dk2-buildroot Retrieving file: /
boot/zImage
7538376 bytes read in 462 ms (15.6 MiB/s)
append: root=/dev/mmcblk0p4 rootwait Retrieving
file: /boot/stm32mp157c-dk2.dtb
117969 bytes read in 148 ms (778.3 KiB/s)
Kernel image @ 0xc2000000 [ 0x000000 - 0x7306c8 ]
#Flattened Device Tree blob at c4000000
```



Generic Distro boot (2)

```
Booting using the fdt blob at 0xc4000000  
Loading Device Tree to cffe0000, end cffffcd0 ... OK  
  
Starting kernel ...
```